BACK MATTER

Appendix A: Case Studies

Five organizations that adopted aspects of specification-driven development, as each of them reported it, with the details that would identify them changed.

Every methodology has to survive contact with the messiness of an organization: the founder who left before writing anything down, the vendor contract signed under deadline pressure, the department that trusts its own spreadsheet more than the shared system. The five cases in this appendix are drawn from a bank's core ledger migration, a clinic network's scheduling overhaul, an e-commerce team's shift away from ad hoc AI use, an enterprise's replacement of a standards document teams ignored, and a government agency's benefits system built in 1995. Between them, they cover most of what the earlier chapters treat one thread at a time: recovering a specification from code that predates everyone currently maintaining it, writing one from a blank page, and using a shared specification to keep several teams, or several technologies, pointed at the same behavior.

▲ WARNING

Before you read these as a track record: each case here is presented as reported by the organization involved, with names, locations, and other identifying details changed. None of the teams involved supplied independently audited figures, and none of these five cases was selected to represent an average outcome.

Read every number in this appendix as what one organization says happened on one engagement, not as a guarantee or a typical result. A processing time cut from 45 days to 5, a review-time reduction of 60 percent, a rewrite that landed at a quarter of its original estimate: each is one organization's report of one engagement, under its own conditions. What you get from the same approach depends on your codebase, your regulatory constraints, and how much specification work your organization is willing to do before it starts generating code.

CASE 1 · FINANCIAL SERVICES

Case 1: COBOL Rewrite Recovered as a Specification

A retail bank, the one whose ledger Chapters 9 and 11 follow, had run its core account system on COBOL written in the late 1980s: deposits, withdrawals, transfers, and account management for 200,000 customers. The architects who designed it had retired. Documentation was outdated. The system worked, but adding a feature took six to eight weeks and risked destabilizing everything around it, while fintech competitors shipped comparable features in a fraction of that time.

The bank's first plan was a straight technology swap: rewrite the COBOL in Java, move to the cloud, an estimated 18 months and $2.5 million. Three months in, the new system's behavior started drifting from the old one. Small differences in rounding. In the order of operations. In how edge cases were handled. Nobody currently at the bank fully understood the original rules; they existed only as COBOL. Some of what looked like rules turned out to be bugs that customers had learned to depend on, which meant fixing them would break customer workflows. The rewrite stalled.

The bank brought in specialists to recover a specification from the code instead of replacing it outright. Over eight weeks, working through the codebase and interviewing the senior staff who still remembered pieces of the original logic, they documented a vision, roughly 200 specific behavior rules, 15 major use cases, and a domain model of 8 core entities. Only then did they generate a Java implementation, validate it against 10,000 test cases pulled from production, and run the old and new systems side by side for three months before cutover.

REPORTED RESULT

- Six months and $600,000, against an 18-month, $2.5 million estimate for the direct rewrite
- Zero production issues at cutover, after three months running in parallel with matching output
- New-feature turnaround down to two to three weeks, from six to eight

The bank's own account credits the result to finally knowing what the system was supposed to do, independent of COBOL or Java. Technology became something they could change without losing the rules underneath it.

CASE 2 · HEALTHCARE

Case 2: Clinic Network Vendor Integration

A healthcare network of 40 clinics and 500 doctors had a different scheduling system at almost every location, each built by a different vendor. Patients couldn't book across clinics. Doctors entered their availability by hand, sometimes into more than one system. Double-bookings were routine, and the network was losing patients to competitors with simpler booking.

The CTO's first move was to buy a single enterprise appointment system: one vendor, one platform, a quoted $3 million and 12 months. Six months into integration, the plan was straining. Every clinic had its own business rules buried in its old workflow. The vendor's platform was flexible enough to accommodate them but complicated enough that different parts of the network configured it differently, and the inconsistencies were multiplying rather than resolving.

Instead of continuing the integration, the network wrote a single specification, shaped like an internal RFC, with clinic managers, doctors, and administrators at the table. It named six use cases (book, cancel, reschedule, manage availability, override, send reminders), a domain model where a patient is unique across the whole network and a doctor's availability lives with the doctor rather than any one clinic, and explicit rules: booking up to 90 days out, specialty-specific slot lengths, reserved emergency slots, 24-hour cancellation notice. Against that one specification, they built five separate implementations, a patient app, a doctor-facing scheduling tool, a clinic admin dashboard, an integration layer, and a backend service, each suited to its own users rather than forced into one shared codebase.

REPORTED RESULT

- Four months and $1.2 million, against the vendor's 12-month, $3 million quote
- Staff time on manual scheduling down 40 percent
- Five new clinics added later by configuring the existing systems, not building new ones

The network's account credits the consistency to the shared specification, not to standardizing on one vendor or one stack. Five different implementations enforced the same rules because all five were built against the same document.

CASE 3 · E-COMMERCE

Case 3: E-Commerce AI Output Inconsistency

An e-commerce company that had grown from a startup to $50 million in revenue had a codebase built by different teams in different eras, each with its own patterns. When they asked AI agents to generate features, the results were inconsistent from one call to the next, with validation logic, naming, and error handling each drawn from a different assumption every time. Developers spent more time fixing the generated code than they would have spent writing it themselves.

The cause was the absence of a shared specification for the AI to work from. Domain models lived in an outdated wiki. Architecture decisions were scattered across old chat threads, and the only place implementation conventions existed at all was in code review comments. Requirements in the issue tracker were often ambiguous too. Every prompt asked the AI to guess at validation, naming, service boundaries, error handling, and testing, and a different guess came back each time.

The team spent two weeks writing it down: a layered architecture (controllers, services, repositories, REST only, Postgres, Redis), domain models for customer, product, cart, order, payment, and inventory, and implementation conventions covering validation, testing, naming, error handling, and logging. Business rules went in too: cart expiry after 24 hours idle, purchases limited to items in stock, refunds processed within 48 hours. The next feature they asked AI to build, gift card purchasing, came back following every one of those conventions, none of which the prompt restated.

REPORTED RESULT

- Feature turnaround down from two to three weeks to three to five days
- Review time down 60 percent
- Sprint velocity up from 12 to 25 story points

The team's own conclusion was that AI output tracked specification clarity directly. Reviewers stopped fixing style and started checking whether the generated code matched business intent, a different job than the one they'd been doing.

CASE 4 · ENTERPRISE ARCHITECTURE

Case 4: Enterprise Standards Replacement

A large enterprise ran dozens of internal systems, built by different teams over different years in Java, Python, Go, and Node.js. Every integration meant reconciling different error handling, different validation, and different data formats, and the architecture team spent more time mediating between teams than helping any one of them ship.

Their first attempt at fixing this was a 150-page architecture standards document and mandatory training. Teams pushed back, arguing that what worked for a payment system didn't fit a scheduling system, and mostly kept using their own patterns. The document went largely unread.

The architecture team changed what it was standardizing. Instead of dictating a technology stack, they wrote four short protocol specifications in RFC style, one each for error handling, logging, authentication, and data validation, fixing only the behavior two systems needed to agree on. Error handling could still be implemented in Java, Python, or Go. Logging could run through ELK, Splunk, or Datadog. Authentication could use Keycloak, Auth0, or Okta. Every system had to implement the four protocols; none was told how.

The reported results here are qualitative rather than numeric. Systems that previously couldn't communicate cleanly began interoperating. Teams kept full control of their own technology choices. New teams could tell what was expected of them without reading 150 pages. Debugging across systems became tractable, because error handling, logging, and validation were now consistent everywhere. Adding a new system stopped requiring an extensive architecture review.

In the architecture team's own description, its role shifted from enforcing rules to maintaining a small number of protocols. That is the same relationship that lets many independent implementations of HTTP interoperate without agreeing on anything else.

CASE 5 · PUBLIC SECTOR

Case 5: Government Benefits Modernization

A government agency's benefits system, built in 1995, handled housing assistance, food programs, healthcare benefits, and education funding. Processing a single application took 45 days, with manual review at every step and constant data-entry errors. Downtime wasn't an option: hundreds of thousands of people depended on the system every day. A 2015 attempt to estimate a full rewrite came back at five years and $80 million, and the agency abandoned it as too risky.

Rather than revive the abandoned rewrite, the agency recovered a specification program by program: reading the existing code, interviewing the benefit officers who still understood the rules, and cross-checking both against current regulation. Each recovered specification (use cases such as apply for benefits and verify eligibility, a domain model of applicant, application, benefit, and verification) was validated with the officers before any new implementation was generated. The old and new systems ran side by side before each cutover.

Four programs went through the process in sequence: housing assistance first, in three months; food programs next, in two; healthcare benefits, the largest and most complex, in three; education funding last, in one month, to consolidate what the team had learned.

REPORTED RESULT

- All four programs done in nine months and about $4 million, against the abandoned $80 million, five-year estimate
- Housing assistance processing time down from 45 days to 5, error rate down from 8 percent to 0.3 percent
- Healthcare benefits, the largest program, processing 15 times faster
- Across all four programs: average processing time down to 3 to 5 days, overall error rate down to about 0.2 percent

The agency's own account credits recovering and validating the existing rules program by program, not any single new technology, and treats each phase as independently tested before the next one began.

READING ACROSS THE FIVE CASES

Common Patterns Across the Five Cases

The patterns below are this book's interpretation of the five cases in this appendix, not a claim about organizations in general. Five cases, however consistent, are not a sample a reader should generalize from.

- ☐ **Specification work is what each organization credited, not a technology choice.** The bank's account is about understanding the ledger's rules, not about Java over COBOL. The enterprise's account is about four protocols, not about which languages its systems happened to use.
- ☐ **Every organization reported spending time up front.** Eight weeks recovering the bank's rules. Two weeks writing the e-commerce team's architecture document. None reported skipping that cost, only earning it back later.
- ☐ **Once a specification existed, more than one team or technology could build against it without drifting apart.** The clinic network shipped five different implementations from one specification. The enterprise let four languages coexist under four protocols.
- ☐ **Where AI was involved, its output tracked how precisely the specification was written.** The e-commerce team reported this directly; the bank's insistence on validating against 10,000 production test cases before cutover is the same idea applied to a higher-stakes system.

Two of the five cases, the bank and the government agency, involved recovering a specification from an undocumented system, rather than writing one from a blank page. In both, the specification was treated as separate from the technology under it once it was recovered, which is why each organization reports having replaced that technology without losing the business rules.

Whether these patterns hold at your organization is a question this appendix cannot answer. It can only report what five specific organizations said happened at theirs.